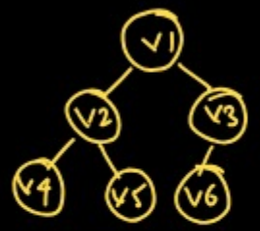
Heaps:

- Heap property has to hold true between every pair of parent & child
  - child's value ≤ parent's value (max heap)
- We <u>can</u> maintain the binary heap as a complete BT. 2 advantages:
  1. Heap can be represented as an array. parent & children's indices can be easily calculated ⇒ no need for a <u>linked structure</u>
  2. ht of complete BT $h = \Theta(\log n)$
     - all ops (deleteMin, insert) $= \Theta(h)$
       $= \Theta(\log n)$



⇔

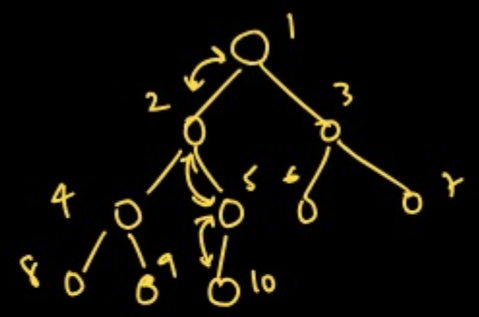
0	1	2	3	4	5	6
-	v1	v2	v3	v4	v5	v6

- Perfect binary tree
  - All levels are full
- Complete binary tree
  - All levels except the last is full & last level is filled from left to right

```
class HeapNode {
    int value;
    HeapNode parent;
    HeapNode left;
    HeapNode right;
}
```

'n' heapnodes ~ 32n bytes

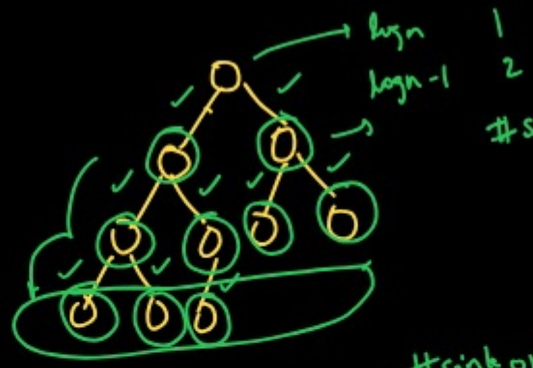
'n' length int array ~ 4n bytes



swim(10)

1. k=10, par=5
2. k=5, par=2
3. k=2, par=1
4. k=1

$\frac{n}{2} \cdot \log n$	$\frac{n}{4} - 1$
$\frac{n}{4} \cdot \log n - 1$	$\frac{n}{8} - 2$
$\frac{n}{8} \cdot \log n - 2$	⋮
⋮	$2 - \log n - 1$
$4 - 2$	$1 - \log n$
$2 - 1$	
$\Theta(n \log n)$	$O(n)$



swim operations = n - 1

↓

heapify using swims = n log n

PBT

nodes in last level = n/2

time for each swim = log n

⇒ heapify ≥ $\frac{n}{2} \log(n)$

sink operations = n/2

↓

O(n)

there is only one node with log n sink time - root

$x \geq y \equiv \sim(x < y)$

x is greater than or equal to y

x is not less than y

if (currnode ≥ maxchildnode)
stop sinking

if (!(currnode < maxchildnode))
stop sinking

$x < y \quad x = y \quad x > y$

$\sim(x > y) \equiv x < y$ or $x = y$

$x \leq y$

Merge sort

- Idea:
  - divide array into 2 halves
  - recursively sort left half & right half
  - merge both halves into a sorted array

merge

sorted: | 3 | 4 | 6 |

sorted: | 1 | 2 | 5 |

| 1 | 2 | 3 | 4 | 5 | 6 |

| 83 | 95 | 110 |

| 54 | 67 | 100 |

| | | | | | |

arr 8 6 5 1 7 3 2 4

MS (arr) → [1, 2, 3, 4, 5, 6, 7, 8]

l: 8 6 5 1

r: 7 3 2 4

✓MS(l) → [1, 5, 6, 8]

l: 8 6

r: 5 1

MS(l) → [6, 8]

l = 8

r = 6

MS(l) → BC

MS(r) → BC

merge(l, r) → [6, 8]

[8] [6]

[6, 8]

MS(R) → [1, 5]

l: 5, r: 1

MS(l) → BC

MS(r) → BC

merge(l, r) → [1, 5]

merge([6, 8], [1, 5]) → [1, 5, 6, 8]

MS(r) → [2, 3, 4, 7]

l = 7 3, r = 2 4

MS(l) → [3, 7]

l = 7 r = 3

MS(7) → BC

MS(3) → BC

merge([7], [3]) → [3, 7]

MS(r) → [2, 4]

l = [2], r = [4]

MS(l) → BC

MS(r) → BC

merge([2], [4])

3 7 2 4

2 3 4 7

merge([3, 7], [2, 4])

merge([1, 5, 6, 8], [2, 3, 4, 7])

```
mergesort(arr):
    l, r = divide(arr)
    mergesort(l) ←
    mergesort(r) ←
    merge(l, r)
```

- $768 = 7\times10^2 + 6\times10^1 + 8\times10^0$

Decimal - base 10 $\quad \underline{700} + \underline{60} + \underline{8}$

$= 768$

- Binary (base 2) to Decimal

$$\overset{4\,3\,2\,1\,0}{11010_2} = 1\times2^4 + 1\times2^3 + 0\times2^2 + 1\times2^1 + 0\cdot2^0$$
$$= 16 + 8 + 0 + 2 + 0$$
$$= 26_{10}$$

Decimal to Binary

26_{10}
$\downarrow -2^4$
10
$\downarrow -2^3$
2
$\downarrow -2^1$
0

$\underset{4}{\underline{1}}\ \underset{3}{\underline{1}}\ \underset{2}{\underline{0}}\ \underset{1}{\underline{1}}\ \underset{0}{\underline{0}}$

- Base 3:
- Base 10 : digits : $\underbrace{0, 1, 2, \dots, 9}_{10 \text{ digits}}$

Base 2 : digits : $\underbrace{0, 1}_{2}$

Base 'b' : digits : $\underbrace{0, 1, 2, \dots, b-1}_{\text{'b' digits}}$

Base 16 (Hexadecimal): $0, 1, 2, \dots, 9, \overset{10}{A}, \overset{11}{B}, \overset{12}{C}, \overset{13}{D}, \overset{14}{E}, \overset{15}{F}$

- Base 3: digits : 0, 1, 2

$$\overset{4\,3\,2\,1\,0}{21021_3} \rightarrow 2\cdot3^4 + 1\cdot3^3 + 0\cdot3^2 + 2\cdot3^1 + 1\cdot3^0$$
$$= 162 + 27 + 0 + 6 + 1$$
$$= 196$$

196
$\downarrow -2\cdot3^4$
34
$\downarrow -1\cdot3^3$
7
$\downarrow -2\cdot3^1$
$1 \xrightarrow{-1\cdot3^0} 0$

$\underset{4}{\underline{2}}\ \underset{3}{\underline{1}}\ \underset{2}{\underline{0}}\ \underset{1}{\underline{2}}\ \underset{0}{\underline{1}}$

Binary to hexadecimal

0 - 0000	4 - 0100	8 - 1000	C - 1100
1 - 0001	5 - 0101	9 - 1001	D - 1101
2 - 0010	6 - 0110	A - 1010	E - 1110
3 - 0011	7 - 0111	B - 1011	F - 1111

9 8 7 6 5 4 3 2 1 0

$\boxed{0010}\boxed{1110}\boxed{1011}_2 = 512 + 128 + 64 + 32 + 8 + 2 + 1$

$= 747$

2 E B_{16}

$= 2 \cdot 16^2 + 14 \cdot 16^1 + 11 \cdot 16^0$

$= 512 + 14 \cdot 16 + 11$

$= 747$

0010 1110 1011_2

0xAB2 + 1 = 0xAB3

0x12 12

$= 1 \cdot 16 + 2$

$= 18$

- 1 byte - 8 bits
  - ↓
  - 2 hex digits

  0xA5

- Unsigned ints :
  - int - 32 bits
  - unsigned int - 32 bits

'k' bits range of unsigned values $[0, 2^k - 1]$

32-bits $[0, 2^{32} - 1]$

bit pattern	unsigned value	signed ints (two's complement)
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	-8
1001	9	-7
1010	10	-6
1011	11	-5
1100	12	-4
1101	13	-3
1110	14	-2
1111	15	-1

'k' bits signed integers range is $[-2^{k-1}, 2^{k-1} - 1]$

1000...0 k-1 0

32bit int $[-2^{31}, 2^{31} - 1]$

16bit short $[-2^{15}, 2^{15} - 1]$

$[-65536, 65535]$

```
int x = 7;
x++;
print(x); //-8
```

-7	1001
+5	0101
-2	1110

```
unsigned x = 15;
x = x + 3;
print(x); //2
```

	111
x	1111
3	0011
	10010

carry discarded

x

0 + 0 = 00

1 + 0 = 01

1 + 1 = 10

1 + 1 + 1 = 11

-8 4 2 1
1 0 1 0

unsigned: $8 + 2 = 10$

signed: $-8 + 2 = -6$

int x = -6; $\boxed{1010}$

(4 bits) int lo = 3 (4 bits) int hi = 7

① $mid = (lo + hi)/2 = (3+7)/2 = -6/2 = -3$

② $\left[mid = lo + (hi - lo)/2 \right]$

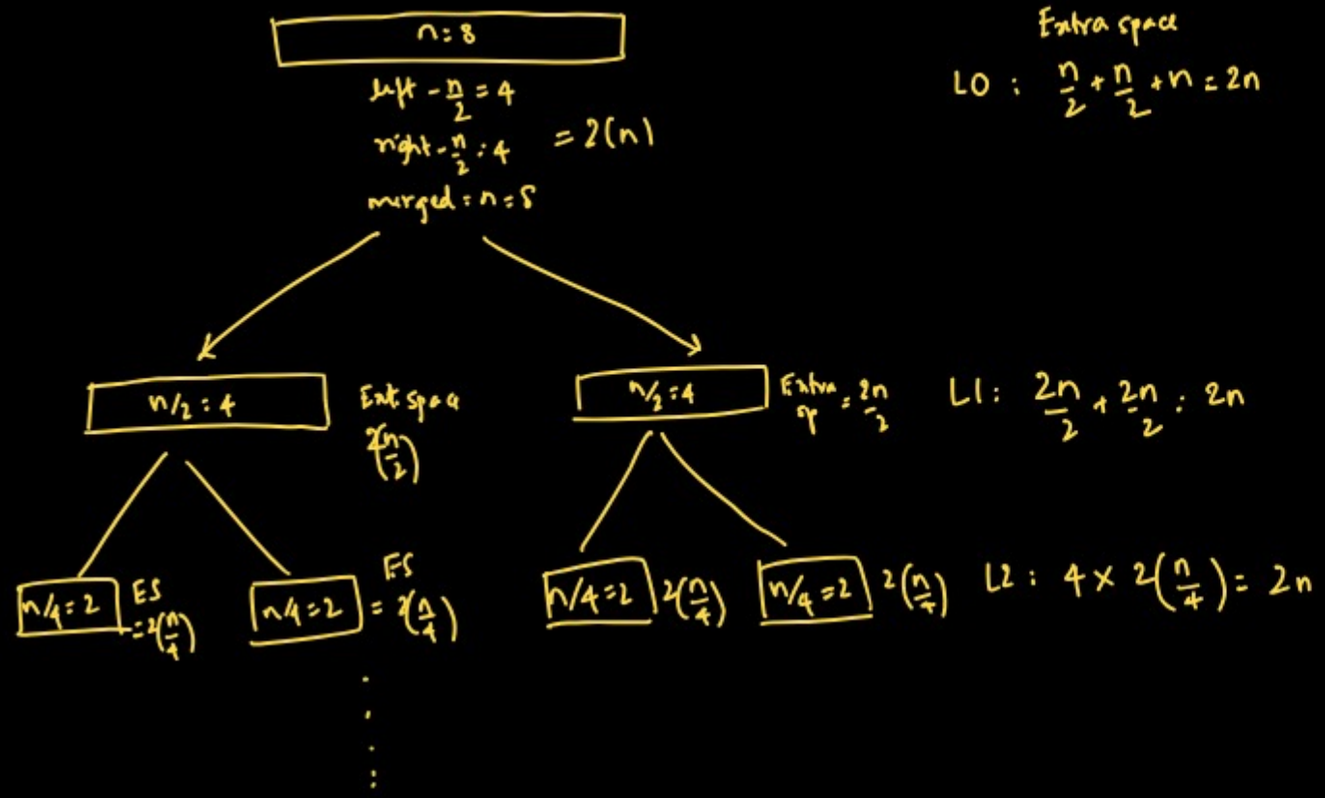
$= 3 + \frac{(7-3)}{2}$

$= 3 + \frac{4}{2}$

$= 5$

long temp = lo + hi

int mid = $\frac{temp}{2}$;



Extra space

L0 : $\frac{n}{2} + \frac{n}{2} + n = 2n$

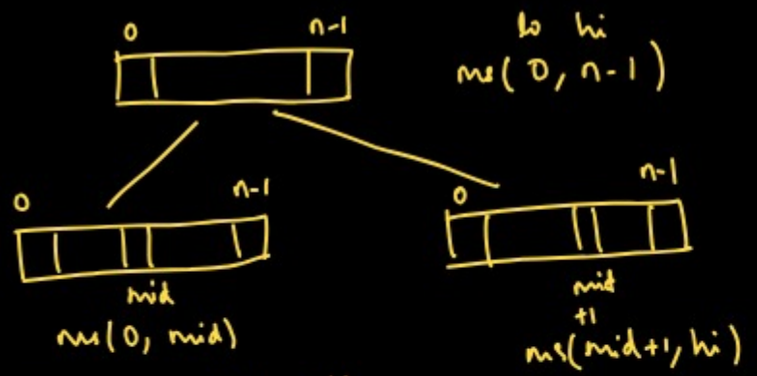
L1 : $\frac{2n}{2} + \frac{2n}{2} = 2n$

L2 : $4 \times 2\left(\frac{n}{4}\right) = 2n$

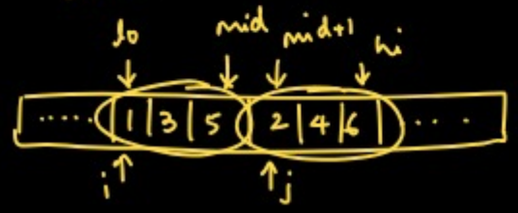
levels = $\log_2 n + 1$

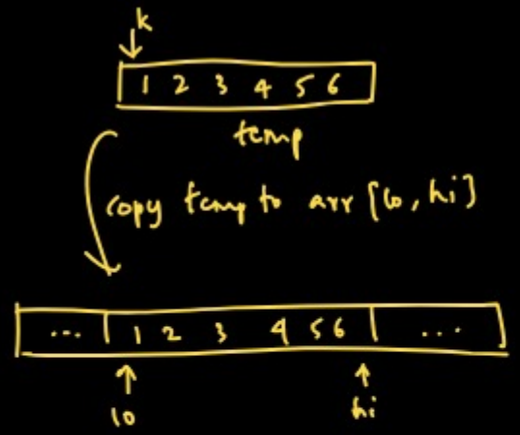
Extra space at each level $= 2n$

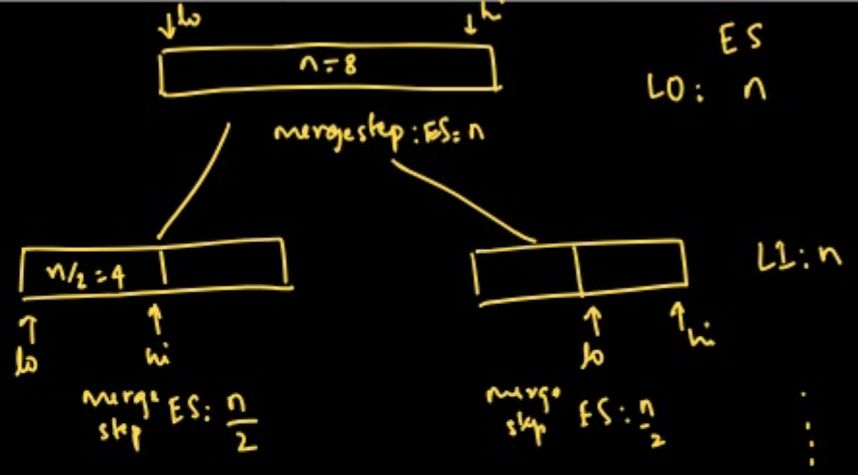
Total Extra space = $\Theta(n \log n)$



after sort(lo, mid)
sort(mid+1, hi)

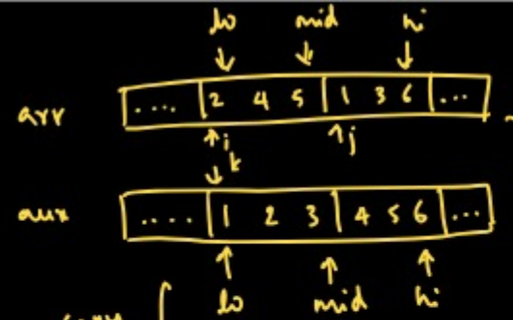






levels: $\log_2 n + 1$

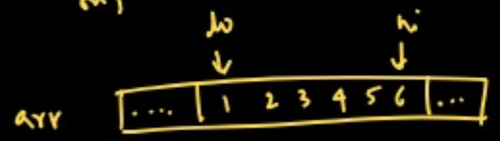
ES at each level $= n$ $\Rightarrow \Theta(n \log n)$ ES

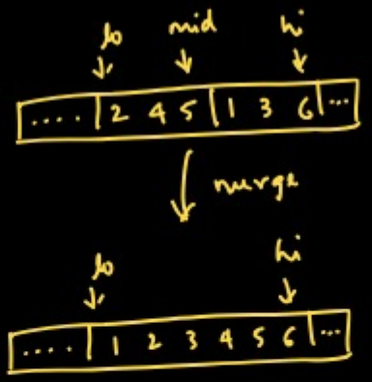


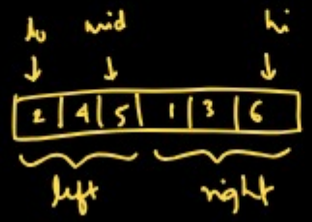
merge step

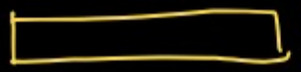
use the aux[lo:hi] instead of temp

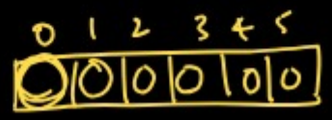
copy over to the orig [lo, hi]

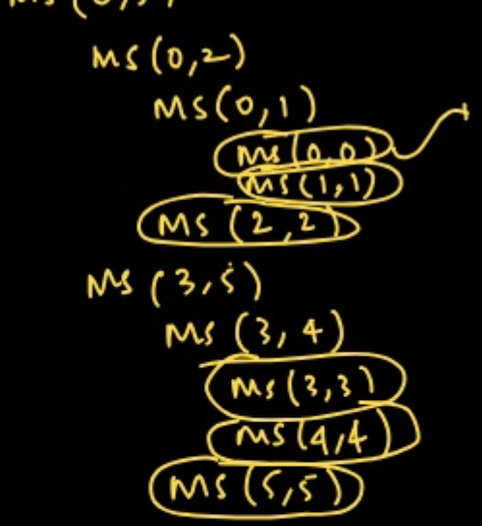


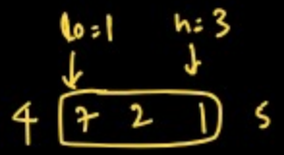












- Whenever we call a function:
  - registers are set of locations very close to CPU & order of magnitude faster than RAM
  - call for f() in main()
    - We'll have store the registers that have values pertaining to main() in RAM
    - Store the return address
    - We need to create a stack frame for the function 'f'.
    - We'll use the registers required & run code in f()
    - Once f is done, jump back to main()
    - Restore the register values from RAM

	Insertion sort ($\Theta(n^2)$)			Merge Sort ($\Theta(n \log n)$)		
Computer	10^3	10^6	10^9	10^3	10^6	10^9
Home (10^8 ops/s)	instant	2.8 hr	317 yr	instant	1s	18 min
Super (10^{12} ops/s)	instant	1s	1 week	instant	instant	instant

- $C(n)$ – total #compares for running merge sort on size 'n' array

mergesort (arr) $\to n$

mergesort (left) $\to$ mergesort of size $n/2$

mergesort (right) $\to$ mergesort of size $n/2$

merge (left, right) $\to$ $\frac{n}{2} \le$ #compares $\le n$

#compares $= n$

$\boxed{4\ 5\ 6}$ $\boxed{1\ 2\ 3}$

$\boxed{1\ 2\ 3\ 4\ 5\ 6}$

$$C(n) = C\left(\frac{n}{2}\right) + C\left(\frac{n}{2}\right) + n$$

$$\boxed{C(n) = 2C\left(\frac{n}{2}\right) + n} \quad \boxed{C(1) = 0}$$

$$\boxed{T(k) = T(k-1) + k} \quad \boxed{T(1) = 1} \Rightarrow \boxed{T(n) = \Theta(n^2)}$$

$$T(n-1) = T(n-2) + (n-1)$$

$$\begin{aligned}
T(n) &= T(n-1) + n \\
&= [T(n-2) + (n-1)] + n \\
&= \underbrace{[T(n-3) + (n-2)]}_{T(n-2)} + (n-1) + n \\
&\vdots \\
&= T(2) + 3 + 4 + 5 + \cdots + (n-1) + n \\
&= [T(1) + 2] + 3 + 4 \cdots + n \\
&= 1 + 2 + 3 + \cdots + n
\end{aligned}$$

$$T(n) = \frac{n(n+1)}{2}$$

$$\boxed{T(n) = \Theta(n^2)}$$

n - power of 2

$$\boxed{C(n) = 2C(n/2) + n} \qquad \boxed{C(1) = 0}$$

$$= 2\left[2C(n/4) + n/2\right] + n$$

$$= 4C(n/4) + n + n$$

$$= 4\left[2C(n/8) + \frac{n}{4}\right] + n + n$$

$$= 8C\left(\frac{n}{8}\right) + n + n + n$$

$$\vdots$$

$$= 2^{\log_2 n - 1}\, C\left(\frac{n}{2^{\log_2 n - 1}}\right) + \underbrace{n + n + \cdots + n}_{\log_2 n - 1 \text{ times}}$$

$$\frac{n}{2^{\log_2 n - 1}} \Updownarrow 2$$

$$1 = \frac{n}{2^{\log_2 n}} \Rightarrow \boxed{2 = \frac{2n}{2^{\log_2 n}} = \frac{n}{2^{\log_2 n - 1}}}$$

$$\frac{2^a}{2^b} = \frac{1}{2^{b-a}}$$

$$C(2) = 2C(1) + 2$$

$$= 2^{\log_2 n - 1}\left(2C(1) + 2\right) + \underbrace{n + n \cdots + n}_{\log_2 n - 1 \text{ times}}$$

$$= 2^{\log_2 n} C(1) + \underbrace{2^{\log_2 n}}_{= n} + \underbrace{n + n + \cdots + n}_{\log_2 n - 1 \text{ times}}$$

$$= n\,C(1) + \underbrace{n + n + \cdots + n}_{\log_2 n \text{ times}}$$

$$C(n) = n(0) + n\log_2 n \Rightarrow \boxed{C(n) = n\log_2 n} \Rightarrow \boxed{C(n) = \Theta(n \log n)}$$

- $\boxed{\begin{array}{l} C(n) = 2C(n/2) + n \\ C(1) = 0 \end{array}}$

Proof by Induction

TP: $\boxed{C(n) = n \log_2 n}$

Base case: $\boxed{n=2} \Rightarrow$

Recurrence relation

$$\Rightarrow C(2) = 2C(2/2) + 2$$
$$= 2C(1) + 2$$
$$= 0 + 2$$
$$= 2$$

Formula

$$C(n) = n \log_2 n$$
$$C(2) = 2 \log_2 2$$
$$= 2 \times 1$$
$$= 2$$

Inductive step

To prove:

If $C(n) = n \log_2 n$, then $C(2n) = 2n \log_2 2n$

$$\text{LHS} = C(2n) = 2C(n) + 2n$$
$$= 2(n \log_2 n) + 2n$$
$$= 2n(\log_2 n + 1)$$
$$= 2n(\log_2 n + \log_2 2)$$
$$= 2n \log_2 (2n)$$
$$= \text{RHS}$$

$\log(ab) = \log a + \log b$

- $S(n) = 1 + 2 + 3 + \cdots + n$

TP: $S(n) = \dfrac{n(n+1)}{2}$

Base case: Formula works for $n = 1$

$S(1) = 1$ by formula

$$S(1) = \frac{1 \times 2}{2} = 1$$

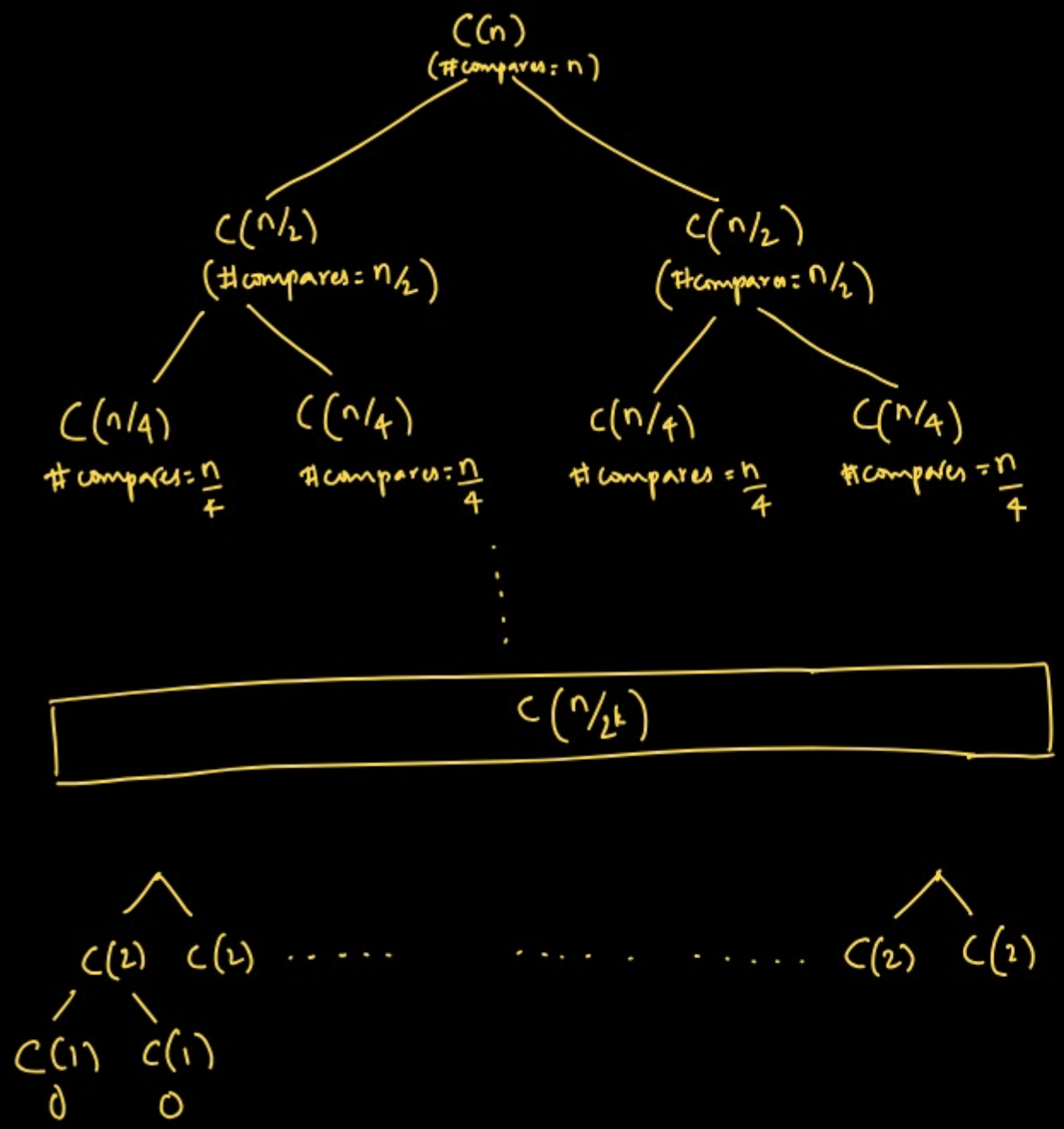
Inductive step

To prove:

If $S(n) = \dfrac{n(n+1)}{2}$, then $S(n+1) = \dfrac{(n+1)(n+2)}{2}$

$$\underbrace{S(n+1)}_{\text{sum of first } n+1 \text{ natural numbers}} = \underbrace{S(n)}_{\text{sum of first 'n' natural numbers}} + (n+1)$$

$$= \frac{n(n+1)}{2} + (n+1)$$
$$= (n+1)\left(\frac{n}{2} + 1\right)$$
$$= \frac{(n+1)(n+2)}{2}$$
$$= \text{RHS}$$



level	# calls	# compares/call	# total compares
0	1	n	n
1	2^1	$n/2^1$	n
2	2^2	$n/2^2$	n
⋮	⋮	⋮	⋮
k	2^k	$n/2^k$	n
⋮	⋮	⋮	⋮
$\log_2 n - 1$	$2^{\log_2 n - 1}$	$\frac{n}{2^{\log_2 n - 1}}$	n

Total levels = $\log_2 n$

Total compares per level = n

$\Rightarrow$ # Total compares $= n \log_2 n$
$= \Theta(n \log n)$